TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

CƠ SỞ TRÍ TUỆ NHÂN TẠO

FUTOSHIKI

Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn

Giảng viên hướng dẫn: Thầy Lê Ngọc Thành
Thầy Huỳnh Lâm Hải Đăng
Thầy Lê Nhựt Nam

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Nguyễn Nhựt Huy	24127398	24C11
Trần Quang Minh Huy	24127403	24C11
Trịnh Khánh Linh	24127437	24C11
Nguyễn Minh Khôi	24127066	24C11

TP. Hồ Chí Minh, tháng 2/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

THÔNG TIN VỀ ĐỒ ÁN

Mã học phần: CSC14003

Tên học phần: Cơ sở trí tuệ nhân tạo

Tên đồ án: Futoshiki (Giải bài toán bằng Logic vị từ bậc nhất và các thuật toán suy diễn)

Hình thức: Bài tập nhóm

Giảng viên: thầy Lê Ngọc Thành, thầy Huỳnh Lâm Hải Đăng, thầy Lê Nhựt Nam

Sinh viên thực hiện:

Nguyễn Nhựt Huy	24127398
Trần Quang Minh Huy	24127403
Trịnh Khánh Linh	24127437
Nguyễn Minh Khôi	24127066

Mục lục

Danh sách hình vẽ **1**

Danh sách bảng **2**

1 Phân công công việc **3**
1.1 Phân công chi tiết . . . 3
1.2 Yêu cầu đồ án . . . 4

2 Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL) **5**
2.1 Giới thiệu . . . 5
2.2 Từ vựng (Vocabulary) . . . 5
2.2.1 Miền giá trị . . . 5
2.2.2 Bảng vị từ . . . 5
2.3 Các tiên đề FOL . . . 5
2.4 Skolem hóa . . . 7
2.5 Chuyển đổi CNF . . . 7
2.5.1 A1 . . . 7
2.5.2 A3 . . . 7
2.5.3 A4 . . . 7
2.6 Kết luận . . . 8
2.7 Xây dựng cơ sở tri thức (Knowledge Base) . . . 8

3 Các thuật toán suy diễn **9**
3.1 Forward Chaining . . . 9
3.1.1 Cơ chế hoạt động . . . 9
3.1.2 Kết hợp Backtracking và heuristic MRV . . . 9
3.1.3 Ví dụ minh họa . . . 10
3.1.4 Nhận xét . . . 11
3.1.5 Mã giả thuật toán . . . 11
3.2 Backward Chaining . . . 13
3.2.1 Cơ chế hoạt động . . . 13
3.2.2 Ứng dụng trong truy vấn giá trị . . . 14
3.2.3 Ví dụ minh họa . . . 14
3.2.4 Nhận xét . . . 15
3.3 Thuật toán A* . . . 15

4 Phân tích heuristic (A*) **16**
4.1 Heuristic 1 – Đếm vi phạm ràng buộc . . . 16
4.1.1 Ý tưởng . . . 16
4.1.2 Định nghĩa . . . 16

4.1.3 Mã giả 16
4.1.4 Chứng minh tính admissible 17
4.1.5 Ví dụ minh họa 17
4.1.6 Nhận xét 17
4.2 Heuristic 2 – Dựa trên AC-3 và miền giá trị 18
4.2.1 Ý tưởng 18
4.2.2 Định nghĩa 18
4.2.3 Mã giả 18
4.2.4 Giải thích 18
4.2.5 Ví dụ minh họa 18
4.2.6 Nhận xét 19

5 Kết quả thực nghiệm **20**

6 So sánh và đánh giá **21**

Danh sách hình vẽ

Danh sách bảng

1.1 Phân công công việc các thành viên 3
1.2 Bảng tự đánh giá yêu cầu bắt buộc 4

Phần 1
Phân công công việc

1.1 Phân công chi tiết

Bảng 1.1: Phân công công việc các thành viên

Thành viên	**Công việc được giao**	**Hoàn thành**
Nguyễn Nhựt Huy	• Quản lý repository nhóm	100%
Trần Quang Minh Huy	• Cài đặt thuật toán A* + heuristic	100%
Trịnh Khánh Linh	• Biểu diễn FOL + Skolem + CNF	100%
Nguyễn Minh Khôi	• Cài đặt thuật toán Brute-force + Backtracking	100%

1.2 Yêu cầu đồ án

Bảng 1.2: Bảng tự đánh giá yêu cầu bắt buộc

STT	Yêu cầu	Trạng thái	Ghi chú
1	Hình thức hóa FOL đầy đủ		Dạng Skolem & CNF cho $\geq$ 3 tiên đề
2	Sinh ground KB tự động theo N		Forward chaining, backward chaining, A*
3	Implement Forward Chaining		Viết từ đầu, đúng trên mọi test case
4	Implement Backward Chaining		Kiểu Prolog, truy vấn Val(i,j,?)
5	Implement A* Search		Heuristic chấp nhận được, có phân tích & so sánh
6	Implement Brute-force & Backtracking		Dùng để so sánh hiệu năng
7	Thiết kế $\geq$ 10 file input		Đa dạng kích thước: 4×4, 5×5, 6×6, 7×7, 9×9
8	Báo cáo đầy đủ		FOL, pseudocode, thực nghiệm (bảng + biểu đồ), so sánh thuật toán
9	Video demo		$\geq$ 3 test case, upload YouTube/Drive
10	Nộp đúng cấu trúc .zip		Docs/, Source/, README.txt, requirements.txt

Phần 2
Biểu diễn bài toán bằng Logic vị từ bậc nhất (FOL)

2.1 Giới thiệu

Futoshiki là một trò chơi giải đố logic được thực hiện trên một bảng kích thước $N \times N$. Nhiệm vụ của người chơi là điền các số từ 1 đến N vào các ô trống sao cho tất cả các ràng buộc của bài toán đều được thỏa mãn. Nhờ cấu trúc rõ ràng và các luật chơi mang tính logic chặt chẽ, Futoshiki thường được sử dụng như một ví dụ điển hình để áp dụng Lôgic Vị từ Bậc Nhất (FOL), vì các ràng buộc này có thể biểu diễn khá trực quan thông qua các mệnh đề có lượng từ.

Trong phần này, báo cáo sẽ trình bày cách xây dựng hệ tiên đề FOL cho bài toán Futoshiki kích thước 4×4. Nội dung bao gồm việc xác định từ vựng sử dụng, xây dựng các tiên đề tương ứng với luật chơi, đồng thời thực hiện Skolem hóa và chuyển đổi một số tiên đề tiêu biểu sang dạng chuẩn CNF.

2.2 Từ vựng (Vocabulary)

2.2.1 Miền giá trị

- Chỉ số hàng, cột: $i, j \in \{1, 2, 3, 4\}$
- Giá trị ô: $v \in \{1, 2, 3, 4\}$

2.2.2 Bảng vị từ

Vị từ	Bậc	Ý nghĩa
$Val(i, j, v)$	3	Ô (i, j) được gán giá trị v
$Given(i, j, v)$	3	Ô (i, j) có giá trị cho sẵn v
$LessH(i, j)$	2	Ràng buộc $<$ ngang giữa (i, j) và $(i, j+1)$
$GreaterH(i, j)$	2	Ràng buộc $>$ ngang giữa (i, j) và $(i, j+1)$
$LessV(i, j)$	2	Ràng buộc $<$ dọc giữa (i, j) và $(i+1, j)$
$GreaterV(i, j)$	2	Ràng buộc $>$ dọc giữa (i, j) và $(i+1, j)$
$Less(v1, v2)$	2	Quan hệ số học $v_1 < v_2$

2.3 Các tiên đề FOL

A1 - Mỗi ô có ít nhất một giá trị

Luật gốc: Mỗi ô trong bảng đều phải được điền một giá trị từ 1 → 4.

$$\forall i \forall j \exists v \; Val(i, j, v)$$

A2 - Mỗi ô có nhiều nhất một giá trị

Luật gốc: Mỗi ô không thể chứa đồng thời hai giá trị.

$$\forall i \forall j \forall v_1 \forall v_2 \ (Val(i,j,v_1) \wedge Val(i,j,v_2)) \Rightarrow v_1 = v_2$$

A3 - Không trùng giá trị trong hàng

Luật gốc: Mỗi hàng là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong hàng trùng nhau.

$$\forall i \forall j_1 \forall j_2 \forall v \ (Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

A3′ - Không trùng giá trị trong cột

Luật gốc: Mỗi cột là một hoán vị của $\{1,2,3,4\}$, tức là không có giá trị nào trong cột trùng nhau.

$$\forall j \forall i_1 \forall i_2 \forall v \ (Val(i_1,j,v) \wedge Val(i_2,j,v) \wedge i_1 \neq i_2) \Rightarrow \perp$$

A4 - Bất đẳng thức ngang $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô trái nhỏ hơn ô phải.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_1,v_2)$$

A4′ - Bất đẳng thức ngang $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i,j+1)$ thì giá trị ô phải nhỏ hơn ô trái.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2)) \Rightarrow Less(v_2,v_1)$$

A5 - Bất đẳng thức dọc $<$

Luật gốc: Nếu có dấu "$<$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô trên nhỏ hơn ô dưới.

$$\forall i \forall j \forall v_1 \forall v_2 \ (LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_1,v_2)$$

A5′ - Bất đẳng thức dọc $>$

Luật gốc: Nếu có dấu "$>$"giữa hai ô (i,j) và $(i+1,j)$ thì giá trị ô dưới nhỏ hơn ô trên.

$$\forall i \forall j \forall v_1 \forall v_2 \ (GreaterV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2)) \Rightarrow Less(v_2,v_1)$$

A6 - Ô cho sẵn

Luật gốc: Các ô được điền sẵn phải giữ nguyên giá trị đó.

$$\forall i \forall j \forall v \ Given(i,j,v) \Rightarrow Val(i,j,v)$$

A7 - Ràng buộc miền giá trị

Luật gốc: Chỉ được điền các số thuộc $\{1, 2, 3, 4\}$*, không được điền số ngoài miền giá trị*

$$\forall i \forall j \forall v \; Val(i, j, v) \Rightarrow (v = 1 \vee v = 2 \vee v = 3 \vee v = 4)$$

2.4 Skolem hóa

Chỉ có A1 chứa lượng từ tồn tại:

$$\forall i \forall j \exists v \; Val(i, j, v)$$

Phân tích cấu trúc lượng từ:

- $\exists v$ nằm trong phạm vi của $\forall i$ và $\forall j$.
- Vì vậy ta thay v bằng hàm Skolem $f(i, j)$ - hàm hai biến phụ thuộc vào i và j.
- Loại bỏ $\exists v$, giữ nguyên $\forall i \forall j$

Sau Skolem hóa:

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$\Rightarrow$ Với mỗi cặp tọa độ (i, j) cụ thể, hàm $f(i, j)$ trả về một giá trị xác định được gán cho ô đó. Tức là, ta không cần biết cụ thể giá trị là gì - chỉ cần biết nó tồn tại và duy nhất ứng với mỗi $\forall i \forall j$.

2.5 Chuyển đổi CNF

2.5.1 A1

$$\forall i \forall j \; Val(i, j, f(i, j))$$

$$Val(i, j, f(i, j))$$

2.5.2 A3

$$\forall i \forall j_1 \forall j_2 \forall v \; (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2) \Rightarrow \perp$$

$$\forall i \forall j_1 \forall j_2 \forall v \; \neg[(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2)]$$

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2)$$

2.5.3 A4

$$\forall i \forall j \forall v_1 \forall v_2 \; (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)) \Rightarrow Less(v_1, v_2)$$

$$\forall i \forall j \forall v_1 \forall v_2 \; \neg[LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j + 1, v_2)] \vee Less(v_1, v_2)$$

$$\neg LessH(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i, j + 1, v_2) \vee Less(v_1, v_2)$$

2.6 Kết luận

Hệ tiên đề FOL gồm 10 tiên đề mô hình hóa đầy đủ luật chơi Futoshiki. Chỉ có một tiên đề cần Skolem hóa (A1), các tiên đề còn lại thuần phổ dụng. Sau khi chuyển sang CNF, hệ có thể sử dụng cho SAT solver hoặc suy diễn logic.

2.7 Xây dựng cơ sở tri thức (Knowledge Base)

Cơ sở tri thức (KB) trong bài toán gồm hai thành phần chính:

- **Facts (Sự thật):** Là các thông tin đúng ngay từ đầu, ví dụ như các ô đã cho sẵn giá trị hoặc các ràng buộc bất đẳng thức trên bảng.
- **Rules (Luật):** Có dạng:

$$p_1 \wedge p_2 \wedge \cdots \wedge p_n \Rightarrow q$$

  trong đó các p_i là điều kiện và q là kết luận.

Ngoài ra, mỗi luật được gắn thêm một biến đếm `count` để theo dõi số lượng điều kiện chưa được thỏa mãn, giúp tối ưu quá trình suy diễn.

Từ dữ liệu đầu vào của bảng $N \times N$, nhóm tự động sinh ra KB dựa trên các loại luật sau:

- **Dữ kiện ban đầu:** Thêm các fact như `Given(i,j,v)` cho các ô đã có sẵn giá trị, và các ràng buộc như `LessH`, `GreaterV`, ...
- **Luật gán giá trị:**

$$Given(i, j, v) \Rightarrow Val(i, j, v)$$

- **Luật ràng buộc hàng và cột:** Nếu một ô đã nhận giá trị v thì các ô khác cùng hàng/cột không được nhận giá trị đó.
- **Luật bất đẳng thức:** Dựa vào các dấu $<, >$ trên bảng để loại trừ các giá trị không hợp lệ của các ô liên quan.

Phần 3
Các thuật toán suy diễn

3.1 Forward Chaining

3.1.1 Cơ chế hoạt động

Thuật toán được cài đặt theo hướng sử dụng **hàng đợi (agenda)** để quản lý các fact cần xử lý. Đồng thời, nhóm sử dụng thêm một cấu trúc chỉ mục (tương tự hash map) để nhanh chóng xác định các luật bị ảnh hưởng khi một fact mới xuất hiện.

Ý tưởng chính của Forward Chaining là lặp lại quá trình suy diễn cho đến khi đạt trạng thái ổn định (không sinh thêm fact mới). Cụ thể:

- Khởi tạo agenda với các fact ban đầu.
- Lặp lại:
  - Lấy một fact từ agenda.
  - Cập nhật các luật liên quan (giảm `count`).
  - Nếu một luật thỏa mãn (`count` = 0), suy ra fact mới.
  - Thêm fact mới vào KB và đưa vào agenda.
- Dừng khi agenda rỗng (đạt điểm cố định).

Sau bước này:

- Nếu tất cả các ô đã được xác định giá trị $\rightarrow$ kết thúc.
- Ngược lại $\rightarrow$ chuyển sang Backtracking để tiếp tục tìm nghiệm.

3.1.2 Kết hợp Backtracking và heuristic MRV

Khi Forward Chaining không thể suy diễn thêm (đạt trạng thái ổn định) nhưng bài toán vẫn chưa giải xong, nhóm sử dụng **Backtracking** để tìm kiếm phần còn lại của nghiệm.

Để giảm không gian tìm kiếm, thuật toán sử dụng heuristic **MRV (Minimum Remaining Values)**. Thay vì chọn ô theo thứ tự cố định, MRV ưu tiên chọn ô có domain nhỏ nhất (tức là ít giá trị khả thi nhất).

Ý tưởng của MRV:

- Ô có domain nhỏ thường bị ràng buộc nhiều hơn.
- Dễ phát hiện mâu thuẫn sớm nếu chọn sai.
- Giảm số nhánh cần duyệt trong cây tìm kiếm.

Quy trình Backtracking kết hợp MRV như sau:

- Chọn ô (i, j) chưa xác định với domain nhỏ nhất.
- Thử lần lượt từng giá trị v trong domain(i, j):
  - Gán $(i, j) = v$ và thêm fact `Val(i,j,v)` vào KB.
  - Chạy lại Forward Chaining để lan truyền ràng buộc.
  - Nếu xảy ra mâu thuẫn (domain rỗng hoặc conflict) thì quay lui và thử giá trị khác.

Nhờ kết hợp Forward Chaining và MRV, thuật toán vừa tận dụng được khả năng suy diễn để thu hẹp domain, vừa đảm bảo tìm được nghiệm đầy đủ với số bước thử ít hơn so với Backtracking thuần túy.

3.1.3 Ví dụ minh họa

Xét bài toán Futoshiki kích thước 3×3 với dữ liệu đầu vào như sau (intput1):

Grid:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ràng buộc ngang:

- $(1,1) < (1,2)$

Ràng buộc dọc:

- $(1,2) > (2,2)$
- $(2,1) > (3,1)$

Bước 1: Khởi tạo KB và Domain

- Từ dữ kiện ban đầu: `Given(1,2,2)` $\Rightarrow$ `Val(1,2,2)`
- Domain ban đầu:

$$(1,2) = \{2\}, \quad \text{các ô còn lại} = \{1,2,3\}$$

Bước 2: Forward Chaining

Từ `Val(1,2,2)`, suy ra:

- Loại bỏ 2 khỏi hàng 1: $(1,1), (1,3)$ không thể là 2
- Loại bỏ 2 khỏi cột 2: $(2,2), (3,2)$ không thể là 2

Xét ràng buộc $(1,1) < (1,2)$ với $(1,2) = 2$:

- $(1,1) < 2 \Rightarrow (1,1) = 1$

Suy ra:

`Val(1,1,1)`

Tiếp tục Forward Chaining:

- Loại 1 khỏi hàng 1 và cột 1

Sau một số bước suy diễn, domain được thu hẹp nhưng vẫn chưa xác định hết các ô.

Bước 3: Áp dụng MRV

Xét các ô chưa xác định, chọn ô có domain nhỏ nhất. Ví dụ:

- Giả sử ô $(2,2)$ còn domain $\{1,3\}$ (nhỏ nhất)

Chọn ô $(2,2)$ để thử giá trị.

Bước 4: Backtracking + Forward Chaining

- Thử $(2,2) = 1$:
  - Thêm `Val(2,2,1)` vào KB
  - Chạy lại Forward Chaining
  - Nếu xảy ra mâu thuẫn $\rightarrow$ quay lui
- Nếu không hợp lệ, thử $(2,2) = 3$:

– Tiếp tục suy diễn và lan truyền ràng buộc

Quá trình này tiếp tục cho đến khi tất cả các ô đều được gán giá trị.

3.1.4 Nhận xét

Forward Chaining giúp lan truyền ràng buộc rất nhanh và giảm đáng kể không gian tìm kiếm ban đầu. Tuy nhiên, do chỉ dựa vào suy diễn nên thuật toán có thể bị dừng sớm khi không còn luật nào áp dụng được.

Vì vậy, việc kết hợp với Backtracking là cần thiết để đảm bảo tìm được nghiệm đầy đủ cho mọi test case.

3.1.5 Mã giả thuật toán

Algorithm 1: FC Solver cho Futoshiki

Input: Dữ liệu bài toán *data*, hàm dừng *stop_check*, cờ ghi log *enable_trace*
Output: Nghiệm *solution* và thống kê *stats*

```
nodes_expanded ← 0 ;
if data.n ≥ 5 then
    enable_trace ← False ;
end if
if enable_trace then
    trace ← [] ;
end if
else
    trace ← ∅ ;
end if
Khởi tạo thời gian time_start ;
Bắt đầu đo bộ nhớ ;
domain ← domain_init(data) ;
kb ← build_kb(data) ;
result ← backtrack(kb, domain, 0, stop_check, trace, enable_trace) ;
Ghi nhận time_end, bộ nhớ sử dụng ;
stats ← {runtime, memory, nodes_expanded, trace} ;
if result = ∅ then
    return (∅, stats) ;
end if
for mỗi ô (i, j) do
    v ← phần tử duy nhất trong result(i, j) ;
    data.grid[i − 1][j − 1] ← v ;
end for
return (data, stats) ;
```

Algorithm 2: Backtracking với FC + MRV

```
Input: kb, domain, depth, stop_check, trace, enable_trace
Output: Domain hoàn chỉnh hoặc ∅
if stop_check() then
    return ∅ ;
end if
nodes_expanded ← nodes_expanded + 1 ;
success ← propagate(kb, domain, depth, stop_check) ;
if NOT success then
    return ∅ ;
end if
Kiểm tra tất cả ô đã xác định chưa ;
if mọi ô chỉ có 1 giá trị then
    return domain ;
end if
cell ← select_mrv(domain) ;
if cell = ∅ then
    return ∅ ;
end if
(i, j) ← cell ;
values ← các giá trị trong domain(i, j) (đã sắp xếp) ;
for mỗi v ∈ values do
    if stop_check() then
        return ∅ ;
    end if
    new_kb ← copy(kb) ;
    new_domain ← deepcopy(domain) ;
    new_domain(i, j) ← {v} ;
    Thêm Val(i, j, v) vào new_kb ;
    result ← backtrack(new_kb, new_domain, depth + 1, stop_check, trace, enable_trace) ;
    if result ≠ ∅ then
        return result ;
    end if
end for
return ∅ ;
```

Algorithm 3: Chọn biến theo MRV

```
Input: domain
Output: Ô (i, j) có domain nhỏ nhất
best_cell ← ∅ ;
best_size ← ∞ ;
for mỗi ô cell do
    values ← domain(cell) ;
    if |values| > 1 và |values| < best_size then
        best_size ← |values| ;
        best_cell ← cell ;
    end if
end for
return best_cell ;
```

Algorithm 4: Propagation (Forward Checking)

```
Input: kb, domain, depth, stop_check
Output: True nếu thành công, False nếu mâu thuẫn
step ← 0 ;
while True do
    if stop_check() then
        return False ;
    end if
    old_domain ← deepcopy(domain) ;
    forward_chaining(kb) ;
    if phát hiện xung đột then
        return False ;
    end if
    Cập nhật domain từ kb ;
    if có domain rỗng then
        return False ;
    end if
    Thêm các singleton vào kb ;
    if domain = old_domain và không có singleton mới then
        break
    end if
    step ← step + 1 ;
end while
return True ;
```

3.2 Backward Chaining

Nếu Forward Chaining là phương pháp suy diễn tiến từ các dữ kiện ban đầu để sinh ra các sự thật mới, thì **Backward Chaining** là cách suy diễn theo hướng ngược lại: bắt đầu từ một mục tiêu cần kiểm tra và truy ngược về các luật và sự thật trong cơ sở tri thức (KB).

Trong đồ án này, Backward Chaining không được dùng để giải toàn bộ bảng, mà chủ yếu đóng vai trò **hỗ trợ truy vấn và kiểm tra tính khả thi** của một giá trị tại một ô cụ thể.

3.2.1 Cơ chế hoạt động

Thuật toán được cài đặt dưới dạng hàm đệ quy `backward_chain(kb, goal, visited, memo)`. Với một mục tiêu cần chứng minh, thuật toán thực hiện:

- Nếu mục tiêu đã tồn tại trong tập `facts` → trả về `True`.
- Ngược lại, tìm các luật có kết luận trùng với mục tiêu thông qua cấu trúc **conclusion index** (giúp truy xuất nhanh thay vì duyệt toàn bộ luật).
- Với mỗi luật tìm được, lần lượt chứng minh các tiền đề của luật bằng cách gọi đệ quy.
- Nếu tất cả tiền đề đều đúng → mục tiêu đúng.

Để tối ưu, thuật toán sử dụng thêm hai kỹ thuật:

- **Loop detection (`visited`):** Tránh lặp vô hạn khi các luật tạo thành vòng.
- **Memoization (`memo`):** Lưu kết quả các mục tiêu đã chứng minh để tránh tính lại nhiều lần.

3.2.2 Ứng dụng trong truy vấn giá trị

Trong bài toán Futoshiki, Backward Chaining được dùng để xác định các giá trị khả thi cho một ô (i, j). Thay vì chứng minh trực tiếp $Val(i, j, v)$, nhóm sử dụng cách tiếp cận đơn giản hơn:

- Với mỗi giá trị $v \in \{1, \ldots, N\}$:
  - Kiểm tra xem có chứng minh được $NotVal(i, j, v)$ hay không.
- Nếu chứng minh được $\rightarrow$ loại v khỏi domain.
- Nếu không chứng minh được $\rightarrow$ giữ v là giá trị khả thi.

Cách làm này giúp loại bỏ các giá trị không hợp lệ mà không cần thử toàn bộ không gian tìm kiếm.

3.2.3 Ví dụ minh họa

Xét bài toán Futoshiki kích thước 3×3 với dữ liệu:

Grid:

$$\begin{bmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ràng buộc:

- $(1, 1) < (1, 2)$
- $(1, 2) > (2, 2)$
- $(2, 1) > (3, 1)$

Ta cần kiểm tra xem ô $(1, 1)$ có thể nhận giá trị 2 hay không.

Bước 1: Thiết lập truy vấn

Thay vì chứng minh trực tiếp $Val(1, 1, 2)$, ta kiểm tra:

$$\texttt{backward_chain}(kb, NotVal(1, 1, 2))$$

Bước 2: Tìm luật liên quan

Từ KB, tồn tại luật ràng buộc hàng:

$$Val(1, 2, 2) \Rightarrow NotVal(1, 1, 2)$$

Bước 3: Chứng minh mục tiêu con

Để chứng minh $NotVal(1, 1, 2)$, ta cần chứng minh:

$$Val(1, 2, 2)$$

Từ dữ kiện đề bài:

$$Given(1, 2, 2) \Rightarrow Val(1, 2, 2)$$

Vì $Given(1, 2, 2)$ là fact $\rightarrow$ suy ra $Val(1, 2, 2)$ đúng.

Bước 4: Kết luận

Suy ra:

$$NotVal(1, 1, 2) \text{ là True}$$

Do đó, giá trị 2 bị loại khỏi domain của ô $(1, 1)$.

Kết luận cuối cùng: Ô $(1, 1)$ **không thể** nhận giá trị 2.

Nhận xét:

Ví dụ trên cho thấy Backward Chaining có thể nhanh chóng loại bỏ các giá trị không hợp lệ bằng cách truy ngược từ mục tiêu. Thay vì thử gán trực tiếp, thuật toán chỉ cần kiểm tra các luật liên quan trong KB, giúp giảm đáng kể số phép thử trong quá trình giải bài toán.

Kết luận: Ô $(1, 2)$ không thể nhận giá trị 1.

3.2.4 Nhận xét

Backward Chaining phù hợp cho các truy vấn cục bộ, đặc biệt khi cần kiểm tra nhanh tính hợp lệ của một giá trị.

Trong khi đó, Forward Chaining hiệu quả hơn trong việc lan truyền ràng buộc trên toàn bộ bảng. Việc kết hợp hai phương pháp giúp hệ thống vừa giảm được không gian tìm kiếm, vừa đảm bảo tính chính xác khi suy diễn.

3.3 Thuật toán A*

A* là một thuật toán tìm kiếm có thông tin (informed search), kết hợp giữa:

- Chi phí đã đi từ trạng thái ban đầu đến trạng thái hiện tại.
- Một hàm ước lượng chi phí còn lại để đạt tới trạng thái đích.

Tại mỗi bước, A* sẽ chọn trạng thái có giá trị nhỏ nhất theo hàm:

$$f(n) = g(n) + h(n)$$

trong đó:

- $g(n)$: chi phí từ trạng thái ban đầu đến trạng thái n.
- $h(n)$: hàm heuristic ước lượng chi phí từ n đến trạng thái đích.

Trong bối cảnh bài toán Futoshiki, mỗi trạng thái tương ứng với một cấu hình của bảng (một phần hoặc toàn bộ các ô đã được gán giá trị). Quá trình tìm kiếm được thực hiện bằng cách:

- Chọn một ô chưa được gán giá trị.
- Thử gán một giá trị hợp lệ từ miền giá trị của ô đó.
- Sinh ra trạng thái mới và đưa vào hàng đợi ưu tiên.

Thuật toán sử dụng một **priority queue** (hàng đợi ưu tiên) để luôn mở rộng trạng thái có giá trị $f(n)$ nhỏ nhất trước. Đồng thời, một tập **visited** được sử dụng để tránh việc lặp lại các trạng thái đã xét.

A* đảm bảo tìm được nghiệm tối ưu nếu hàm heuristic thỏa mãn tính chất admissible. Tuy nhiên, hiệu quả của thuật toán phụ thuộc rất lớn vào chất lượng của heuristic được sử dụng.

Phần 4
Phân tích heuristic (A*)

4.1 Heuristic 1 – Đếm vi phạm ràng buộc

4.1.1 Ý tưởng

Heuristic1 được xây dựng dựa trên việc kiểm tra các ràng buộc bất đẳng thức trong bảng Futoshiki.
Ý tưởng chính là:

- Duyệt qua từng ô (i, j) trong bảng.
- Kiểm tra tất cả các ràng buộc liên quan đến ô đó (ngang và dọc).
- Nếu tồn tại ràng buộc:
  - bị vi phạm, hoặc
  - chưa xác định (tồn tại ô có giá trị 0),

  thì ô đó được xem là **vi phạm**.

Giá trị heuristic là tổng số ô vi phạm. Trạng thái có ít vi phạm hơn sẽ được ưu tiên mở rộng.

4.1.2 Định nghĩa

$$h(n) = \text{số ô vi phạm ràng buộc}$$

4.1.3 Mã giả

```
FUNCTION Heuristic1(state, data):
    count ← 0

    FOR mỗi ô (i, j):
        violated ← FALSE

        FOR mỗi ràng buộc liên quan đến (i, j):
            IF ràng buộc không thỏa hoặc có ô chưa gán:
                violated ← TRUE
                BREAK

        IF violated:
            count ← count + 1

    RETURN count
```

4.1.4 Chứng minh tính admissible

Gọi:

- V là tập các ô bị vi phạm
- $|V|$ là số lượng ô vi phạm

Ta có:

- Mỗi ô vi phạm cần được xử lý ít nhất một lần để thỏa mãn ràng buộc.
- Mỗi bước gán chỉ có thể thay đổi một ô.

Do đó, chi phí tối thiểu để đạt nghiệm thỏa mãn:

$$h^*(n) \geq |V|$$

Trong khi heuristic:

$$h(n) = |V|$$

Suy ra:

$$h(n) \leq h^*(n)$$

$$\Rightarrow \text{Heuristic1 là admissible}$$

4.1.5 Ví dụ minh họa

Xét bài toán 3×3:

```
Grid:
0   2   0
0   0   0
0   0   0
```

Các ràng buộc:

- $(1,1) < (1,2)$
- $(1,2) > (2,2)$
- $(2,1) > (3,1)$

Áp dụng heuristic:

Các ô vi phạm gồm:

$$(1,1), (1,2), (2,1), (2,2)$$

Do đó:

$$h(n) = 4$$

4.1.6 Nhận xét

- Ưu điểm: đơn giản, dễ cài đặt, admissible.
- Nhược điểm: đánh giá còn thô, chưa phản ánh chính xác mức độ gần nghiệm.

4.2 Heuristic 2 – Dựa trên AC-3 và miền giá trị

4.2.1 Ý tưởng

Heuristic2 sử dụng thuật toán lan truyền ràng buộc **AC-3** để thu hẹp miền giá trị (domain) của các biến trước khi đánh giá trạng thái.

Ý tưởng:

- Áp dụng AC-3 để loại bỏ các giá trị không hợp lệ.
- Nếu tồn tại biến có domain rỗng → trạng thái không khả thi.
- Ngược lại, đếm số biến vẫn còn nhiều hơn một giá trị khả dĩ.

4.2.2 Định nghĩa

$$h(n) = \begin{cases} \infty & \text{nếu tồn tại domain rỗng} \\ \text{số biến có } |domain| > 1 & \text{ngược lại} \end{cases}$$

4.2.3 Mã giả

```
FUNCTION Heuristic2(state, data):
    domains ← copy(state.domains)

    IF AC3(domains) == FALSE:
        RETURN Infinity

    count ← 0
    FOR mỗi domain d:
        IF size(d) > 1:
            count ← count + 1

    RETURN count
```

4.2.4 Giải thích

Sau khi áp dụng AC-3:

- Nếu một domain bị rỗng, điều đó cho thấy trạng thái hiện tại không thể dẫn đến nghiệm hợp lệ.
- Nếu tất cả domain đều hợp lệ, heuristic sẽ đo số lượng biến chưa được xác định (có nhiều hơn một giá trị).

Do đó:

- Giá trị heuristic càng nhỏ → trạng thái càng gần nghiệm.
- Giá trị lớn → còn nhiều biến chưa chắc chắn.

4.2.5 Ví dụ minh họa

Sau khi áp dụng AC-3 cho ví dụ trên, ta có:

Ô	Domain	Size
(1,1)	{1}	1
(1,2)	{2}	1
(1,3)	{1,2,3}	3
(2,1)	{1,2,3}	3
(2,2)	{1}	1
(2,3)	{1,2,3}	3
(3,1)	{1,2,3}	3
(3,2)	{1,2,3}	3
(3,3)	{1,2,3}	3

Các ô có $|domain| > 1$ gồm:

$$(1,3), (2,1), (2,3), (3,1), (3,2), (3,3)$$

Do đó:

$$h(n) = 6$$

4.2.6 Nhận xét

- Ưu điểm:
  - Sử dụng AC-3 giúp phát hiện sớm các trạng thái vô nghiệm.
  - Đánh giá chính xác hơn mức độ "chưa hoàn chỉnh" của trạng thái.
- Nhược điểm:
  - Chi phí tính toán cao hơn do phải chạy AC-3.

Phần 5
Kết quả thực nghiệm

Phần 6
So sánh và đánh giá